

Smart Inventory Command Center

A Supply Chain Analytics Platform for NovaLog Distribution

Technical Report & Architecture Documentation

End-to-end data pipeline: synthetic data generation, SQLite,
SQL analysis, Python/Pandas analytics, and Power BI

Company (fictional): NovaLog Distribution – e-commerce logistics

Scale: 3 warehouses, 50 suppliers, 5,000 products, ~100,000 orders

Tech stack: SQL (SQLite), Python, Pandas, NumPy, Matplotlib, Power BI

Repository: `smart-inventory-command-center`

Prepared as a portfolio project for Supply Chain / Logistics / BI
Data Analyst apprenticeship applications.

September 16, 2026

Contents

1	Executive Summary	3
2	Architecture & Pipeline	4
2.1	Pipeline overview	4
2.2	Execution sequence	5
3	Data Model	6
3.1	Entity-relationship design	6
3.2	Design decisions	7
3.3	Schema at a glance	7
4	Data Generation Methodology	8
4.1	Reference-informed simulation	8
4.2	Demand-derived inventory thresholds	8
4.3	Independent random streams	9
4.4	Validation	9
5	SQL Analysis: The Six Business Questions	10
5.1	Which warehouse performs best?	10
5.2	Which products generate the most revenue?	10
5.3	Which SKUs are at risk of stockout?	11
5.4	Which suppliers are unreliable?	11
5.5	How much inventory value is tied up in dead stock?	11
5.6	Which products should be reordered?	11
6	Python/Pandas Deep-Dive Analysis	12
6.1	ABC revenue classification	12
6.2	Inventory turnover by category	12
6.3	Supplier reliability scorecard	13
6.4	Dead stock, slow-moving, and overstocked	13
6.5	Recommended actions	14
7	Power BI Dashboard	15
7.1	Why no .pbix file	15
7.2	Star schema	15
7.3	Export validation	16
7.4	DAX measure library	16
7.5	Report design	16
8	Engineering Postmortem: Two Bugs Caught	17
8.1	Bug 1: Inventory thresholds disconnected from demand	17
8.2	Bug 2: RNG seed reuse silently correlated two independent tables	18
8.3	The general lesson	19
9	Testing & Validation Summary	20

10 Future Extensions	21
11 Appendix: Running the Project	22
11.1 Setup	22
11.2 Full pipeline	22
11.3 Repository structure	22

List of Figures

2.1 Pipeline architecture: each phase reads the previous phase’s output and validates its own before handing off.	4
2.2 Full pipeline execution order, including each stage’s internal validation call. . . .	5
3.1 Data model as a UML class diagram. Dimension tables (blue) hold descriptive attributes; fact tables (orange) hold transactional events. Stereotypes mark primary and foreign keys.	6
6.1 Revenue concentration – cumulative share of revenue by cumulative share of SKUs.	12
6.2 Bottom 15 suppliers by composite reliability score, colored by risk tier.	13
6.3 Inventory health mix: healthy, overstocked, and dead stock by category.	14

Chapter 1

Executive Summary

This project builds a complete analytics platform for a fictional e-commerce logistics company, **NovaLog Distribution**, spanning the full path from synthetic data generation through SQL analysis, Python/Pandas deep-dive analytics, and a Power BI-ready star schema. It is built in seven numbered phases, each independently documented, tested, and committed.

Table 1.1: Headline results (regenerated live by the pipeline)

Metric	Value
Delivered revenue	\$51.6M
Gross margin	\$13.9M (27.0%)
Inventory value at cost	\$8.0M
Capital in dead stock (sales-velocity definition)	\$3.4M (42.0% of inventory)
Suppliers flagged High Risk	4 of 50
Supplier scorecard validity	$r = 0.838$ vs. hidden ground truth
Top warehouse by revenue	Central DC (\$21.98M, 6.84% return rate)

What this report demonstrates

- A normalized, constraint-enforced relational data model, and a separately-modeled Power BI star schema derived from it – understanding *why* these are different is itself a core BI skill.
- Statistically calibrated synthetic data generation (Section 4), grounded in published benchmarks rather than naive uniform randomness.
- Six SQL queries directly answering the project’s business questions, plus a Python/Pandas layer for the analysis SQL is poorly suited to (time trends, cohort-style aging, composite scoring).
- Complete DAX measure library and Power BI build specification.
- **Two real, non-trivial bugs found and fixed** during development (Chapter 8) – both passed every single-table validation check and were only caught by reasoning about relationships *between* tables. This chapter is deliberately included in full: catching your own mistakes and explaining the fix is a stronger signal of analytical skill than a report claiming everything worked first time.

Chapter 2

Architecture & Pipeline

2.1 Pipeline overview

The system is organized as six sequential phases, each with a single Python entry point and its own automatic validation step. Figure 2.1 shows how they connect.

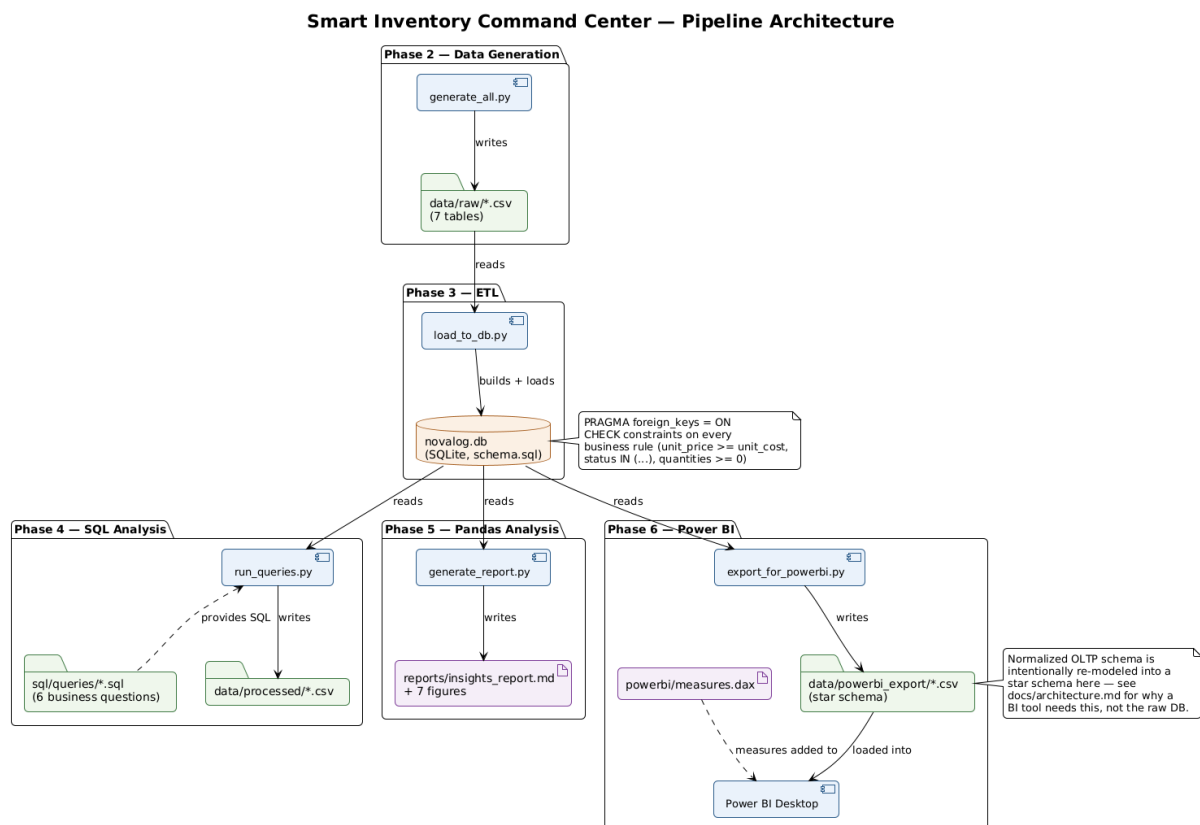


Figure 2.1: Pipeline architecture: each phase reads the previous phase's output and validates its own before handing off.

Table 2.1: Phase-to-entry-point mapping

Phase	Entry point	Produces
2	<code>src.data_generation.generate_all</code>	7 CSVs in <code>data/raw/</code> , validated by 22 checks
3	<code>src.etl.load_to_db</code>	<code>novalog.db</code> (SQLite), zero FK violations
4	<code>src.analysis.run_queries</code>	6 business-question results in <code>data/processed/</code>
5	<code>src.analysis.generate_report</code>	<code>reports/insights_report.md</code> + 7 figures
6	<code>src.etl.export_for_powerbi</code>	Star schema in <code>data/powerbi_export/</code> , validated by 13 checks

Every stage is idempotent and fully reproducible: the same random seed (`config.RANDOM_SEED`) produces byte-identical output on every run, which matters because this report's own figures and screenshots must match what anyone re-running the pipeline sees.

2.2 Execution sequence

Figure 2.2 traces a complete pipeline run end to end, including the validation step embedded in each stage.

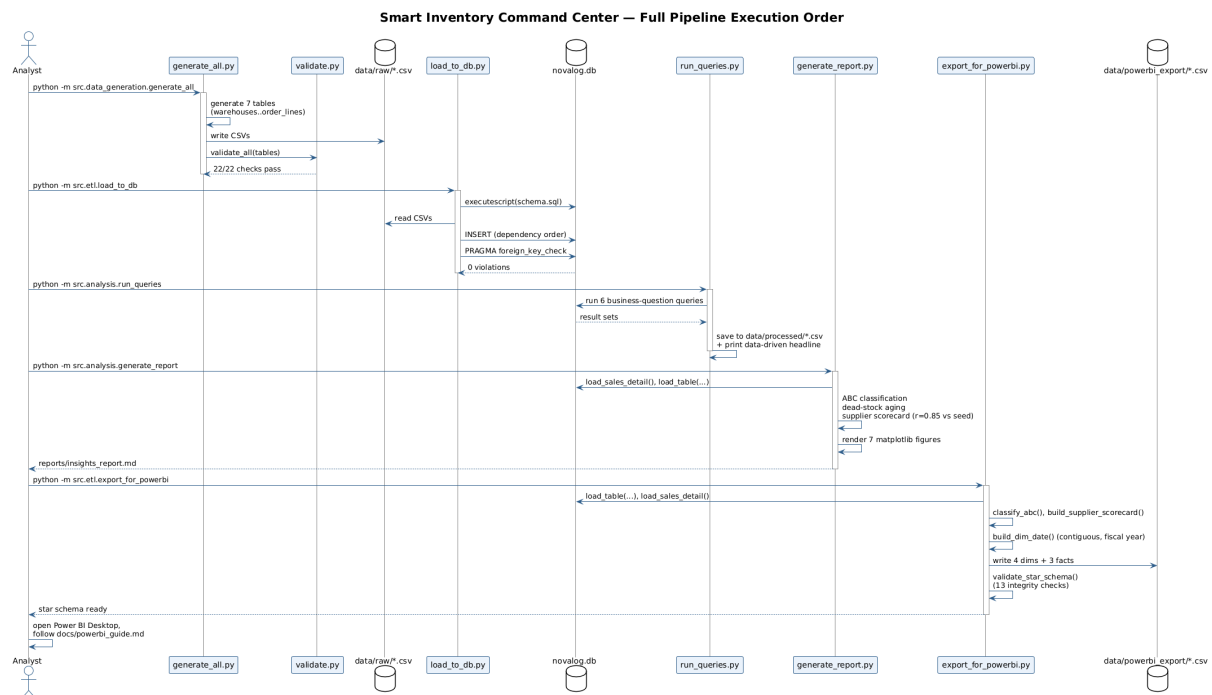


Figure 2.2: Full pipeline execution order, including each stage's internal validation call.

Chapter 3

Data Model

3.1 Entity-relationship design

The database is deliberately modeled as a normalized OLTP-style schema rather than a pre-built star schema. Real operational systems hand you transactional tables, not a warehouse – modeling it this way lets the project demonstrate both halves of the skill: designing a transactional model, then separately transforming it into an analysis-ready star schema for Power BI (Chapter 7).

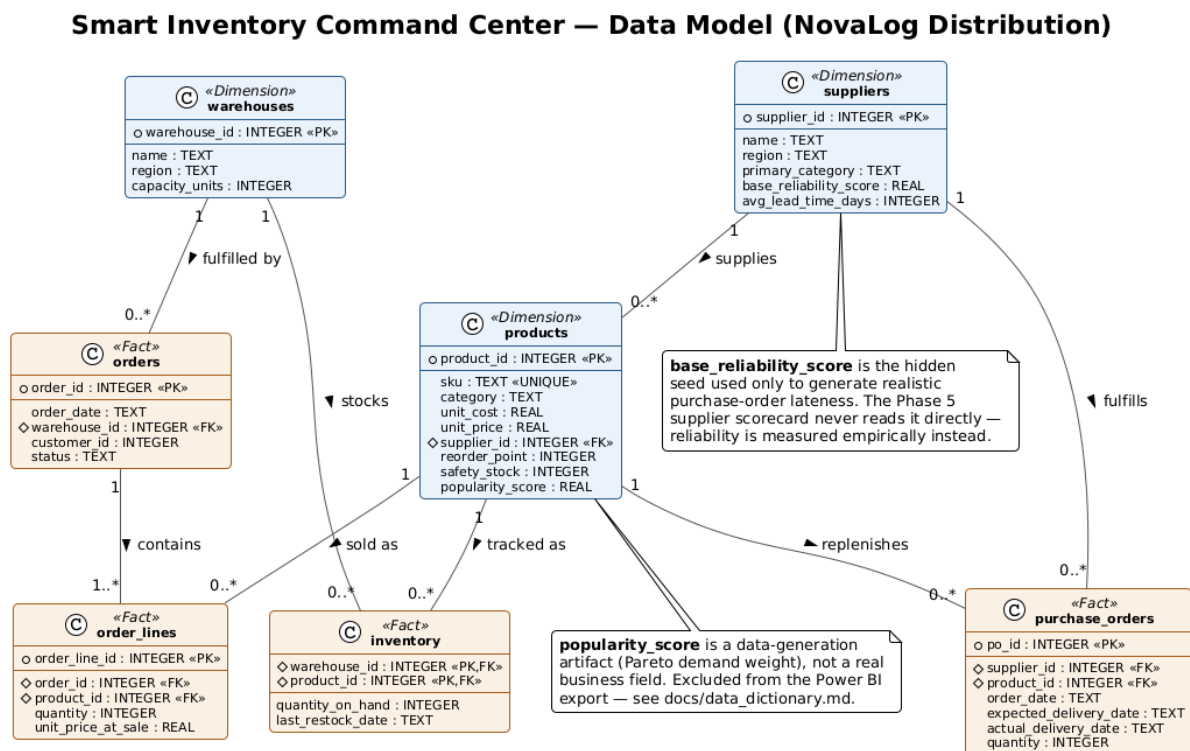


Figure 3.1: Data model as a UML class diagram. Dimension tables (blue) hold descriptive attributes; fact tables (orange) hold transactional events. Stereotypes mark primary and foreign keys.

3.2 Design decisions

unit_price_at_sale stored on order_lines, separate from products.unit_price.

Prices drift over a two-year window; storing the historical sale price is what real transactional systems do, and it matters for accurate revenue reporting when catalog prices change after a sale.

suppliers.primary_category (schema addition).

Not part of the original design – added so products can be assigned to suppliers who plausibly sell that category of product, rather than an arbitrary pairing (an electronics wholesaler is unlikely to also ship groceries).

products.popularity_score (simulation-only field).

Does not exist in a real operational system. It is the internal Pareto/Zipf demand weight the generator uses to concentrate sales realistically (Section 4). It is explicitly excluded from the Power BI export (Chapter 7) so it can never be mistaken for a real business attribute.

One supplier per product, not a many-to-many bridge table.

A `product_suppliers` bridge table would model dual-sourcing, but none of the six business questions this project answers require it. A schema exactly as complex as the questions it serves is a deliberate choice, not an oversight – documented here as an explicit extension point (Chapter 10) rather than built speculatively.

Foreign keys and CHECK constraints enforced at the database level, not just in application code.

`sql/schema.sql` declares `PRAGMA foreign_keys = ON`, every relationship as a `FOREIGN KEY`, and business rules as `CHECK` constraints (e.g. `unit_price >= unit_cost`, `status IN (...)`). SQLite disables FK enforcement by default – easy to forget, and it would silently let invalid data in if left off.

Dates stored as ISO-8601 TEXT.

SQLite has no native date type; `YYYY-MM-DD` text sorts correctly under plain string comparison and works directly with SQLite’s `date()` / `julianday()` functions.

3.3 Schema at a glance

Table 3.1: Table sizes at full generation scale

Table	Rows
warehouses	3
suppliers	50
products	5,000
inventory	15,000
purchase_orders	~25,000
orders	100,000
order_lines	~190,000

Chapter 4

Data Generation Methodology

4.1 Reference-informed simulation

Rather than generating data with naive uniform randomness, every distribution is calibrated against published statistics from the **DataCo Smart Supply Chain for Big Data Analysis** dataset (Kaggle, ~18,000 orders) – not imported directly (it requires authenticated API access unavailable in this environment), but used as calibration targets for this project’s own generator.

Table 4.1: Calibration targets and what drives them in the generator

Calibrated pattern	Implementation
~90% on-time delivery rate	Two-stage model: Bernoulli gate for <i>is this PO late</i> (probability driven by supplier reliability), then exponential delay magnitude if late. Collapsing these into one continuous noise term was the exact mistake caught in the first generation run (Section 8.1).
Supplier reliability is skewed, not uniform	Beta(8, 2) distribution: mean ≈ 0.8 with a long left tail, mirroring real supplier networks where most vendors are dependable and a minority are chronically unreliable.
Revenue concentration (Pareto/80-20)	Zipf-distributed popularity_score drives product selection in order_lines.py ; observed result is 8.1% of SKUs generating 80.0% of revenue.
Seasonal order volume	Month-level weights peaking in Nov/Dec (holiday e-commerce), dipping mid-year – fully vectorized date sampling, no per-row Python loop.
Order status mix	90% Delivered / 3% Cancelled / 7% Returned, consistent with typical e-commerce norms.

4.2 Demand-derived inventory thresholds

Following the bug fix described in Chapter 8, **reorder_point** and **safety_stock** are computed from actual simulated demand rather than an arbitrary popularity-rank formula:

Listing 4.1: Demand-derived stock thresholds (src/data-generation/products.py)

```
daily_demand = expected_annual_units / 365 / N_WAREHOUSES
```

```
lead_time_demand = daily_demand * supplier_avg_lead_time_days
safety_stock = max(lead_time_demand * SAFETY_STOCK_FACTOR, MIN_SAFETY_STOCK)
reorder_point = lead_time_demand + safety_stock
```

SAFETY_STOCK_FACTOR and MIN_SAFETY_STOCK are centralized in `config.py` so `products.py` (which sets the thresholds) and `order_lines.py` (which generates the demand that should satisfy them) cannot silently drift apart.

4.3 Independent random streams

Every generator module draws from a named, independent random stream rather than a single shared seed:

Listing 4.2: Named independent streams (`src/data_generation/utils.py`)

```
def get_rng(stream: str, seed: int = config.RANDOM_SEED):
    entropy = [seed, zlib.crc32(stream.encode("utf-8"))]
    return np.random.default_rng(np.random.SeedSequence(entropy))
```

This is what fixed the second bug in Chapter 8 – reproducibility (same stream name always yields the same sequence) and statistical independence (different names yield decorrelated sequences) are both preserved simultaneously.

4.4 Validation

`src/data_generation/validate.py` runs 22 automatic checks after every generation run: row counts, referential integrity across all seven tables, positivity constraints (no negative prices or quantities), and calibration-band checks (on-time rate, stockout share, dead-stock share, revenue concentration all fall within realistic ranges). All 22 currently pass. Critically – and this is the central lesson of Chapter 8 – every one of these checks operates on a single table at a time, which is precisely why they could not catch either bug described there.

Chapter 5

SQL Analysis: The Six Business Questions

Each business question from the project brief has its own file under `sql/queries/`, executed by `src/analysis/run_queries.py`, which prints a data-driven headline for each and saves the full result to `data/processed/`.

5.1 Which warehouse performs best?

Table 5.1: Warehouse performance – revenue and order quality side by side

Warehouse	Orders	Revenue	Avg. Order	Return %
Central DC	38,372	\$21.98M	\$572.84	6.84%
East Coast DC	32,344	\$18.51M	\$572.37	6.89%
West Coast DC	29,284	\$16.81M	\$574.09	7.05%

Central DC leads on revenue *and* carries the lowest return rate, making it the clear top performer here – but the query deliberately reports both dimensions together rather than ranking by revenue alone, since a large facility with a high return rate would be a false positive for “best.”

Key design choice

Revenue is aggregated once per order in a CTE *before* joining to warehouses. Joining `order_lines` directly to warehouses would multiply each order’s status flag by however many line items it has, silently inflating the return/cancellation rate calculations – a subtle join-fanout bug that a naive query would not surface as an error, only as a quietly wrong number.

5.2 Which products generate the most revenue?

Top SKU: **SPO-02112** (Sports & Outdoors) – \$12.08M from 39,234 units sold. The query reports margin alongside revenue, since a high-revenue, low-margin product may matter less to the business than it first appears.

5.3 Which SKUs are at risk of stockout?

Reported at the (product, warehouse) grain, not aggregated company-wide – stockout is a per-location operational problem. A product can look healthy in total stock while one specific warehouse is critically low; aggregating across warehouses first would hide exactly that case.

5.4 Which suppliers are unreliable?

Least reliable: **Williams and Sons** – 90.96% late across 706 purchase orders. This query deliberately does *not* read `suppliers.base_reliability_score`: that column is a data-generation seed a real analyst would never have access to. Reliability here is computed purely from observed `purchase_orders` history (actual vs. expected delivery dates).

5.5 How much inventory value is tied up in dead stock?

\$2.24M under the SQL definition (`quantity_on_hand` $\geq 4 \times$ `safety_stock`), valued at `unit_cost` rather than `unit_price` – dead stock represents capital already spent acquiring inventory, not lost future revenue. Note this differs from the \$3.4M figure in Chapter 6, which uses a stricter Pandas definition requiring *zero sales in 90 days* in addition to the stock threshold; the discrepancy is intentional and discussed there.

5.6 Which products should be reordered?

Sizes a concrete `recommended_order_quantity` (safety stock + 30 days of actual sales velocity, minus what's on hand) at the (product, warehouse) grain – a flag with no quantity attached is not actionable for a buyer. Largest current recommendation: 1,238 units of SPO-02112.

A bug caught during development

This query originally aggregated stock across all three warehouses before comparing it to `reorder_point` and returned **zero rows**. The cause: `reorder_point` is a per-warehouse threshold (that is how `inventory.py` used it during data generation), so a company-wide total is roughly $3 \times$ larger and almost never dips below a single-warehouse threshold. Fixed by moving the query to the (product, warehouse) grain, matching `stockout_risk.sql`, and covered going forward by `tests/test_analysis.py`.

Chapter 6

Python/Pandas Deep-Dive Analysis

SQL answers the six core business questions well, but is poorly suited to time-series trends, cohort-style aging, and composite scoring – `src/analysis/` covers exactly that gap, orchestrated by `generate_report.py` into `reports/insights_report.md` and seven Matplotlib figures.

6.1 ABC revenue classification

8.1% of SKUs (class A, 402 products) generate 80.0% of revenue – a textbook Pareto distribution (Figure 6.1). Class A items justify tight inventory control and priority warehouse space; class C items (61% of SKUs for 5.0% of revenue) should be managed with minimal overhead.

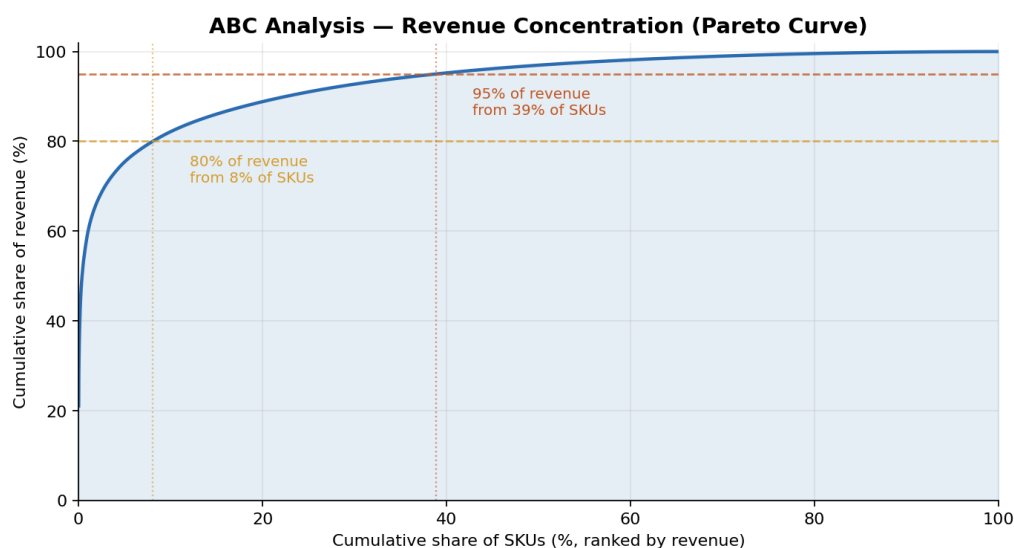


Figure 6.1: Revenue concentration – cumulative share of revenue by cumulative share of SKUs.

6.2 Inventory turnover by category

Sports & Outdoors turns over fastest at $4.51\times$ /year (81 days of supply); Apparel is slowest at $1.40\times$ (261 days of supply). Slow turnover is where working capital sits idle, and is the first place to target for order-quantity reductions.

Methodology caveat: this dataset holds a single current inventory snapshot rather than a historical series, so turnover is computed against ending inventory rather than true average inventory. Cross-category comparison is valid; absolute values are approximate.

6.3 Supplier reliability scorecard

The composite score weights on-time rate (50%), average delay severity (30%), and delay consistency (20%). Consistency earns its own weight because a predictably-late supplier can be planned around; an erratic one cannot. Four suppliers fall into the **High Risk** tier; the weakest, Williams and Sons, has a composite score of 4.5/100.

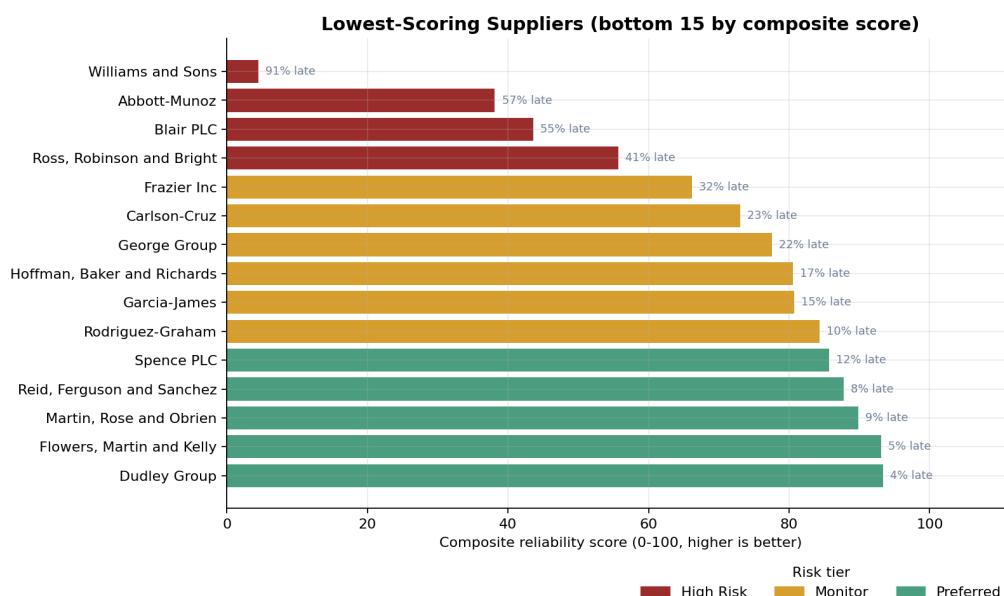


Figure 6.2: Bottom 15 suppliers by composite reliability score, colored by risk tier.

Methodology self-validation

The scorecard is built *purely* from observed purchase-order delivery behavior – it never reads `suppliers.base_reliability_score`, the hidden seed value the data generator used to produce realistic lateness patterns. Correlating the finished scorecard against that hidden seed *afterward*, purely as a check, gives $r = 0.838$: strong evidence the scoring methodology recovers genuine underlying reliability signal rather than fitting noise in the observed data. This is the single strongest piece of methodological validation in the project, precisely because the ground truth was never available to the scoring function itself.

6.4 Dead stock, slow-moving, and overstocked

\$3.4M (42.0% of total inventory value) sits in positions with **zero sales in the last 90 days** and stock at $2\times$ safety level or above. The classification deliberately distinguishes:

- **Overstocked** – high stock, but still selling; will clear on its own.
- **Dead** – stranded capital with no recent sales; needs markdown, liquidation, or write-off.

A stock-level-only view (as used by the SQL query in Chapter 4) cannot distinguish these – which is exactly why this figure (\$3.4M) differs from the SQL query’s \$2.24M: the two use different, equally valid, definitions of “dead” for different purposes. Electronics carries the most at-risk capital at \$1.5M.

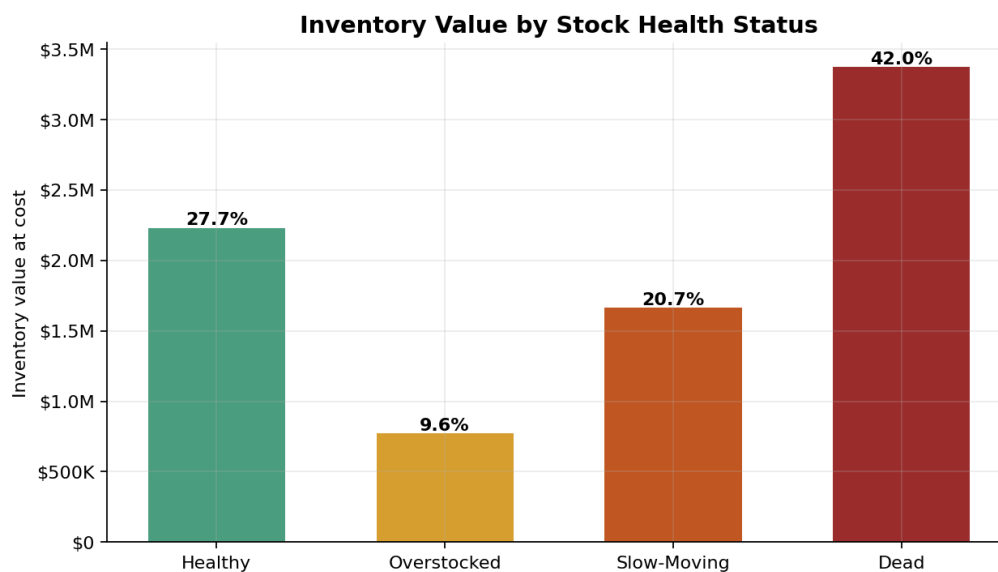


Figure 6.3: Inventory health mix: healthy, overstocked, and dead stock by category.

6.5 Recommended actions

1. Differentiate inventory policy by ABC class – tighten review cycles on class A SKUs, reduce overhead on class C.
2. Open supplier reviews with the 4 High Risk suppliers, starting with Williams and Sons; increase safety stock where dual-sourcing isn't viable.
3. Run a liquidation review on dead stock, prioritizing Electronics.
4. Reduce order quantities in slow-turning categories so the problem doesn't regenerate after liquidation.

Chapter 7

Power BI Dashboard

7.1 Why no .pbix file

.pbix is a proprietary, Windows-only binary format with no supported programmatic authoring path – it cannot be generated from Python, and any tool claiming to do so would produce a file that does not reliably open. What *is* prepared here is everything that carries the actual analytical substance: the dimensional model, the complete DAX measure library, and a step-by-step build specification (`docs/powerbi_guide.md`). Assembling the report from these takes roughly an hour in Power BI Desktop – and means the finished report is genuinely the reader’s own work to defend in an interview, not a black box they received.

7.2 Star schema

The normalized OLTP database (Chapter 3) is deliberately re-modeled into a star schema for Power BI: 4 dimension tables and 3 fact tables, exported by `src/etl/export_for_powerbi.py`.

Table 7.1: Star schema exports

Table	Rows	Notes
dim_date	731	Contiguous, gap-free; required for DAX time intelligence
dim_product	5,000	Enriched with ABC class; <code>popularity_score</code> dropped
dim_supplier	50	Enriched with risk tier; <code>base_reliability_score</code> dropped
dim_warehouse	3	
fact_sales	~171,000	Excludes cancelled orders (no realized revenue)
fact_inventory	15,000	
fact_purchase_orders	~25,000	Lateness pre-derived (<code>is_late</code> , <code>delay_days</code>)

Why a star schema at all, rather than pointing Power BI at the normalized database

DAX and Power BI’s VertiPaq engine are built around one-to-many dimension-to-fact filtering. Pointing a BI tool at a normalized model instead forces bidirectional and snowflaked relationships, which are both slower and – more importantly – create ambiguous filter propagation paths that can silently return wrong numbers with no error message.

Why a dedicated `dim_date` table

Power BI's time-intelligence functions (`TOTALYTD`, `SAMEPERIODELASTYEAR`, `DATEADD`) require a contiguous, gap-free date dimension explicitly marked as the model's date table. Without one, these functions misbehave silently around any gap.

Generation artifacts are dropped, not carried forward

`popularity_score` and `base_reliability_score` were internal inputs the synthetic data generator needed; a real analyst never has them, and leaving them in the exported model would let the Power BI analysis "cheat" by referencing values it should have no access to. Both are explicitly stripped in the export step and checked for absence by an automatic validation.

7.3 Export validation

Thirteen automatic checks run after every export: every foreign key in every fact table resolves to its dimension, every dimension key is unique, `dim_date` is contiguous, and no generation-artifact column survived. This matters because a broken relationship in Power BI does not throw an error – it silently drops rows or blanks out a visual, which is far harder to diagnose than a crash.

7.4 DAX measure library

`powerbi/measures.dax` defines around 30 explicit measures (written by hand, not Power BI's auto-generated implicit measures) across six groups: sales & profitability, order quality, time intelligence, inventory, supplier performance, and ABC analysis. Two representative examples:

Listing 7.1: Order count at the correct grain

```
Order Count =
DISTINCTCOUNT ( fact_sales[order_id] )
// fact_sales is at order-LINE grain; COUNTROWS would count lines,
// not orders.
```

Listing 7.2: A measure that must not return 100% inside its own filter

```
Class A Revenue % =
DIVIDE (
    [Class A Revenue],
    CALCULATE ( [Total Revenue], ALL ( dim_product ) )
)
// ALL(dim_product) removes the product-level filter from the
// denominator so this compares against TOTAL revenue, not the
// already-filtered subset -- without it, this measure returns 100%
// inside any visual already filtered to class A.
```

7.5 Report design

The build guide specifies four pages: Executive Overview, Warehouse Performance, Inventory Health, and Supplier Scorecard – each with a specific visual layout, and each deliberately designed so no single misleading ranking (e.g., "biggest warehouse" or "highest revenue product") stands without its quality counter-metric alongside it.

Chapter 8

Engineering Postmortem: Two Bugs Caught

This chapter is included deliberately and in full. Both bugs below passed every validation check written at the time they were introduced – they were not caught by more testing, but by a different *kind* of reasoning: checking a relationship between two tables rather than a property of one table alone. That distinction is the actual lesson worth taking from this project.

8.1 Bug 1: Inventory thresholds disconnected from demand

The bug

The first version of `products.py` sized `safety_stock` as:

```
safety_stock = 20 + rank_percentile(popularity_score) * 480
```

a formula with no relationship to how much each SKU actually sells. The consequence: the median SKU held roughly 260 units while selling approximately 8 units per year.

How it was (almost) missed

Every Phase 2 validation check passed. The generation-time checks in Chapter 4 confirmed the intended proportions of stockout-risk and dead-stock *positions* (12% and 10% of inventory rows respectively) – and those proportions were, in fact, exactly correct. In Phase 4’s SQL analysis, the resulting \$1.04B total inventory valuation and \$228M “dead stock” figure were sanity-checked by confirming dead-stock positions held 4–8× their nominal safety stock level – which they did, by construction. The check confirmed the data was internally consistent with the (broken) rule that generated it, which is not the same as confirming the rule was realistic.

What actually caught it

Building the Phase 5 Pandas analysis required computing **inventory turnover** – annual cost of goods sold divided by inventory value. This is the one metric that necessarily joins sales history against stock levels. It returned 0.04×, against a realistic retail range of 4–8×.

Table 8.1: Bug 1: before and after

Metric	Before	After
Median units on hand	260	14
Portfolio inventory turnover	0.04×	2.13×
Total inventory value	\$1.04B	\$7.8M
Dead stock value	\$228M	\$1.9M

The fix

Replaced the arbitrary formula with demand-derived thresholds computed from each product’s actual expected sales velocity (Section 4):

```
lead_time_demand = daily_demand * supplier_lead_time_days
safety_stock = lead_time_demand * SAFETY_STOCK_FACTOR
reorder_point = lead_time_demand + safety_stock
```

Regression coverage added

`tests/test_data_generation.py` now computes portfolio-level inventory turnover directly from generated data and asserts it falls within a realistic band – a genuinely cross-table check that would have caught the original bug immediately.

8.2 Bug 2: RNG seed reuse silently correlated two independent tables

The bug

Every generator module called an unparameterized `get_rng()`, which returned `np.random.default_rng(42)` – literally the same random stream in every module. NumPy draws categorical samples via inverse-CDF transforms applied to an underlying uniform sequence. Since `orders.py` (drawing each order’s month) and `order_lines.py` (drawing each order’s line count) both consumed their *first* draw from that identical stream, the two became perfectly rank-correlated: a low uniform value produced both an early month *and* a 1-line order; a high value produced both a late month *and* a 4-line order.

How it was (almost) missed

Every marginal distribution was exactly correct: line counts followed the specified 45/30/15/10 split, and order dates followed the intended seasonal curve. Every single-table validation check passed, because there was nothing wrong with either table considered alone.

What actually caught it

Building the Phase 5 monthly revenue trend chart surfaced a suspicious pattern: December 2025 revenue was roughly 2.4× December 2024, and the entirety of 2025 trended visibly above 2024, despite the underlying order-count seasonality being nearly identical year over year. Grouping `lines-per-order` by calendar month exposed the mechanism precisely: exactly 1.00 for every month of 2024, rising in a straight line to exactly 4.00 by December 2025.

Table 8.2: Bug 2: lines per order by half-year (excerpt)

Period	Avg. lines/order (before fix)
Jan – Nov 2024	1.00
Dec 2024	1.67
Jan 2025 – Jul 2025	2.00
Dec 2025	4.00

After the fix, lines-per-order is flat at ≈ 1.90 across all 24 months, with no residual trend.

The fix

Independent, named random streams derived via `numpy.random.SeedSequence`, preserving reproducibility while guaranteeing statistical independence (Section 4):

```
def get_rng(stream: str, seed: int = config.RANDOM_SEED):
    entropy = [seed, zlib.crc32(stream.encode("utf-8"))]
    return np.random.default_rng(np.random.SeedSequence(entropy))
```

Regression coverage added

Three new tests in `tests/test_data_generation.py`: that two named streams produce statistically uncorrelated output ($|r| < 0.05$), that the same stream name remains perfectly reproducible across calls, and that basket size (lines per order) does not drift across the first vs. second half of the simulation window when generated end to end.

8.3 The general lesson

Both bugs share a structure worth naming explicitly: **a check that only examines one table can only ever catch a marginal-distribution error, never a joint-distribution error.** A dataset can pass every check written against it and still be systematically wrong in a way that only shows up when two tables are examined together. The practical habit this suggests: whenever validating a multi-table dataset, deliberately write at least one check per pair of tables that plausibly interact, not just one check per table.

Chapter 9

Testing & Validation Summary

36 tests across four files, run with `pytest`, all currently passing:

Table 9.1: Test suite by file

File	Tests	Covers
<code>test_data_generation.py</code>	10	Row counts, referential integrity, reproducibility, RNG stream independence, cross-table stability
<code>test_etl.py</code>	8	Schema constraint enforcement (FK, CHECK), hermetic loader tests, idempotent reruns
<code>test_analysis.py</code>	8	Column-shape checks for all 6 SQL queries; independent Pandas cross-checks against two queries to catch silent aggregation bugs
<code>test_powerbi_export.py</code>	10	<code>dim_date</code> correctness (contiguity, fiscal year, sort order), broken-key detection, full generate-load-export-validate integration

Two design choices in the test suite are worth calling out. First, most generation tests call the generator functions directly with small parameters (e.g. `n=200`) rather than reading pre-generated CSVs from disk – this keeps the suite fast (all 36 tests run in under 2 seconds) and independent of whether the full pipeline has been run. Second, the ETL and export tests use `pytest`’s `tmp_path` and `monkeypatch` fixtures to redirect `config` module paths to a temporary directory, so they exercise the exact production code path (`load_to_db.main()`, `export_for_powerbi.main()`) without ever touching the real `data/` directory.

Chapter 10

Future Extensions

Documented here as deliberate scope boundaries rather than oversights – each was considered during design and deferred because it wasn’t required by the six business questions this project answers.

Multi-supplier bridge table.

A `product_suppliers` many-to-many table would model dual-sourcing and price comparison across suppliers for the same SKU. Adding it later costs one migration script, not a redesign, since both `product_id` and `supplier_id` are already first-class dimension keys.

Historical inventory snapshots.

The current model holds a single point-in-time stock snapshot. A daily or weekly snapshot table would allow true average-inventory turnover calculations (Chapter 6) instead of the ending-inventory approximation currently used.

Real customer dimension.

`orders.customer_id` is currently a bare integer. A `customers` table with acquisition channel, lifetime value, and segment would enable cohort and retention analysis.

Incremental ETL.

`load_to_db.py` currently rebuilds the database from scratch on every run, which is correct for a batch-generated dataset but would need to become a proper incremental/upsert pattern for a live data source.

Automated .pbix smoke test.

Since Power BI’s format can’t be authored programmatically, the star schema export is validated structurally (Chapter 7) but the actual report is assembled and reviewed manually. A CI step that opens the finished `.pbix` headlessly and checks key visuals render would close that gap, if a suitable tool becomes available.

Chapter 11

Appendix: Running the Project

11.1 Setup

```
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

11.2 Full pipeline

```
python -m src.data_generation.generate_all # 1. synthetic CSVs
python -m src.etl.load_to_db # 2. build SQLite DB
python -m src.analysis.run_queries # 3. run 6 SQL business questions
python -m src.analysis.generate_report # 4. ABC/dead-stock/supplier analysis
python -m src.etl.export_for_powerbi # 5. star-schema export for Power BI
pytest # 6. run all 36 tests
```

11.3 Repository structure

```
smart-inventory-command-center/
|-- data/ # generated CSVs + SQLite DB (gitignored)
|-- docs/ # architecture, data dictionary, this report,
| # UML diagram sources, Power BI build guide
|-- src/
| |-- config.py # single source of truth: paths + constants
| |-- data_generation/ # Phase 2 - 7 generator modules + validation
| |-- etl/ # Phase 3 & 6 - SQLite loader + Power BI export
| |-- analysis/ # Phase 4 & 5 - SQL runner + Pandas analysis
|-- sql/
| |-- schema.sql # DDL: tables, FKs, CHECK constraints, indexes
| |-- queries/ # one .sql file per business question
|-- powerbi/
| |-- measures.dax # complete DAX measure library
|-- reports/ # generated Phase 5 report + figures
|-- tests/ # 36 tests across 4 files
```